Smart HAS — Documentação do Projeto (Fase 5)

> **Como usar este arquivo:** ele é a base do PDF de entrega. Preencha os campos do grupo, cole prints e o link do vídeo, e exporte para PDF (Word → "Salvar como PDF", ou VS Code + extensão *Markdown PDF*).

---

1. Identificação do grupo

Nome completo	RM	Foto
Giovanna Cardoso Pinheiro Cury Marques	554433	*(inserir foto)*

- **Curso / Turma:** *(preencher)*
- **Vídeo de apresentação (YouTube, não listado):** *(colar link)*
- **Repositório GitHub:** *(colar link, se aplicável)*

---

2. Visão geral

O **Smart HAS** é um sistema inteligente de monitoramento urbano (sensores, alertas e áreas de risco) alinhado à **Sociedade 5.0**, com a camada **AI Logistics Extension**. Nesta fase, o sistema evoluiu para uma solução **integrada em três camadas** que compartilham a **mesma API REST**:

- **Mobile (Flutter):** app de campo para operadores.
- **Back-end (Spring Boot):** API REST + banco de dados, o "cérebro" do sistema.
- **Web (Angular):** dashboard administrativo.

---

3. Parte 1 — Escolha da stack e justificativa (Opção B: Flutter)

3.1 Decisão

Optamos por **manter e aprimorar significativamente a solução Flutter** (Opção B), em vez de migrar para React Native.

3.2 Justificativa técnica

- **Base já consolidada:** o app Flutter já existia; migrar para React Native significaria refazer telas sem agregar valor arquitetural — o esforço foi melhor investido em **evolução técnica real** (arquitetura, integração, testes).
- **Um único codebase multiplataforma:** Flutter entrega Android, iOS, Web e Desktop a partir do mesmo código, o que reduz custo de manutenção — coerente com um produto que precisa escalar.
- **Ecossistema maduro** para o que o projeto exige: HTTP/REST, gerência de estado (Provider), persistência local (shared_preferences) e Material 3.
- **Alinhamento com o back-end:** a camada de serviços do app foi desenhada para consumir exatamente a API REST do Spring Boot (Parte 2), a mesma do dashboard Angular (Parte 3).

3.3 Alinhamento com os objetivos do projeto

O Smart HAS visa **apoiar decisões** com dados de sensores em tempo real. A refatoração tornou o app **confiável e conectado**: estados de carregamento/erro explícitos, autenticação com sessão e integração com dados reais — requisitos de um sistema de monitoramento que embasa decisões.

3.4 Roadmap tecnológico

- **[OK] Concluído (fases anteriores):** protótipo Flutter (login, dashboard, alertas e mapa) com dados estáticos.
- **[EM ANDAMENTO] Nesta fase:**
- Flutter: refatoração para **arquitetura em camadas**, camada de rede, autenticação com sessão persistida, novas funcionalidades e testes.
- Spring Boot: **API REST** com JWT, banco de dados, CRUD, validação, Swagger.
- Angular: **dashboard administrativo** consumindo a mesma API.
- **[PROXIMO] Próximas fases:** deploy em nuvem com HTTPS, notificações push (FCM), mapa nativo (Google Maps) com geolocalização e a camada de IA (AI Logistics Extension) analisando os dados dos sensores.

---

4. Parte 1 — Aprimoramentos do app Flutter

Pasta: `mobile-flutter/`

4.1 Arquitetura (antes → depois)

- **Antes:** telas com dados fixos, `Provider` com lista de `String`, serviço apontando para uma API de exemplo, arquivos vazios, dependências não usadas.
- **Depois: arquitetura em camadas** com separação clara de responsabilidades:

```
lib/
├── core/        config, network (ApiClient + exceções), state, theme, mock
```

```
├── models/        User, Sensor, Alert (fromJson/toJson + enums)
├── services/      Auth/Sensor/Alert Service (chamam a API)
├── repositories/  abstração + fallback offline + persistência do token
├── providers/     estado (auth, sensor, alert, settings)
├── screens/       login, home, alertas, mapa, detalhe do sensor
└── widgets/       componentes reutilizáveis
```

4.2 Melhorias implementadas (valor agregado)

- **Camada de rede robusta:** `ApiClient` com headers, JWT Bearer, timeout e **exceções tipadas** (`NetworkException`, `UnauthorizedException`, ...).
- **Integração real com a API REST** + **fallback offline** e flag de ambiente (`useMockData`) para alternar mock ↔ API sem alterar a UI.
- **Autenticação com sessão persistida** (auto-login) via `shared_preferences`.
- **Estados de UI completos:** loading, erro com "tentar novamente", vazio e sucesso.
- **Novas funcionalidades:** dashboard com KPIs, *pull-to-refresh*, busca de alertas, **formulário validado para criar alerta**, detalhe do sensor, mapa que plota sensores por coordenadas.
- **Material 3** com tema claro/escuro alternável e persistido.
- **Padrões e boas práticas:** injeção de dependências (composition root), Repository Pattern, imutabilidade dos models, código comentado.
- **Testes** de parsing de models e de widget (`flutter test`).

---

5. Parte 2 — Back-end escalável com Spring Boot

Pasta: `backend-springboot/`

5.1 Necessidade real atendida

Centralizar e servir, com segurança, os dados de **sensores** e **alertas** do Smart HAS para o app mobile e o dashboard web, com controle de acesso por perfil.

5.2 Arquitetura e boas práticas

- **Camadas:** `model` (entidades JPA) → `repository` (Spring Data) → `service` (regra de negócio) → `controller` (REST) + `dto` (contratos de entrada/saída) + `config`/`exception` (infra).
- **API RESTful** cobrindo todas as operações do app (CRUD de sensores e alertas + login + resumo do dashboard).
- **Autenticação e autorização:** Spring Security + **JWT**; senhas com BCrypt; autorização por **role** (`ADMIN`/`OPERATOR`).
- **Persistência:** banco **H2** (via JPA/Hibernate), com dados iniciais (seed). Operações **CRUD** otimizadas pelos repositórios Spring Data.

- **Validação e tratamento de erros:** Bean Validation nas requisições + `@RestControllerAdvice` retornando erros padronizados `{message, status, timestamp}`.
- **Comunicação segura:** CORS controlado + Bearer token em todas as chamadas.
- **Spring MVC + Thymeleaf:** página inicial server-side em `/` com status da API.
- **Documentação: Swagger/OpenAPI** (springdoc) em `/swagger-ui.html`.
- **Testes:** JUnit + MockMvc (carga de contexto, login e proteção de rotas).

5.3 Endpoints (resumo)

Ver tabela completa em `docs/API_CONTRACT.md`. Principais:
`POST /api/auth/login`, `GET/POST/PUT/DELETE /api/sensors`,
`GET/POST /api/alerts`, `PUT /api/alerts/{id}/resolve`, `GET /api/dashboard/summary`.

---

6. Parte 3 — Integração web com Angular

Pasta: `web-angular/`

6.1 O que foi feito

Painel administrativo que consome **a mesma API REST** do app mobile.

- **Comunicação:** `HttpClient` em serviços dedicados (`AuthService`, `SensorService`, `AlertService`) + **interceptor** que injeta o JWT.
- **Rotas:** `/login`, `/home` (dashboard) e `/admin` (gestão de alertas), com **guard de autenticação**.
- **Data binding (todos os tipos exigidos):** interpolação `{{ }}`, *property binding* `[ ]`, *event binding* `( )` e **two-way** `[( )]`.
- **Diretivas:** `*ngIf` (loading/erro/condicional) e `*ngFor` (listas de sensores e alertas).
- **Formulário funcional com `[(ngModel)]`:** login e criação de alertas.
- **Estilo e feedback:** cards, tabela, *badges* de status coloridos e mensagens de sucesso/erro.

---

7. Como executar (resumo)

1. **API:** `cd backend-springboot && mvn spring-boot:run` → http://localhost:8080
2. **Web:** `cd web-angular && npm install && npx ng serve` → http://localhost:4200
3. **App:** `cd mobile-flutter && flutter pub get && flutter run`

Login: `admin` / `1234`. Swagger: http://localhost:8080/swagger-ui.html

---

8. Conclusão

O Smart HAS evoluiu de um protótipo de telas estáticas para uma **solução integrada, segura e escalável**, com app mobile, API REST e dashboard web compartilhando o mesmo contrato — pronta para receber a camada de IA nas próximas fases.